Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

School of Computing

Department of Computer Science and Engineering

USE CASE REAL LIFE EXAMPLE

Program- Department Name	**B.Tech – CSE**
Academic Year/Semester	2026-27 / Summer
Faculty Name	Dr. S. Hemamalini
Course Code/Course Name	10211CS227 / Problem solving and Testing
Slot No & Date	S3-2 / 22-10-2026
Student Name	AVULA SURENDRA
Title of the Complex Engineering problem / problems	SMART PARKING MANAGEMENT SYSTEM
Description / Complexity Addressed	The Smart Parking Management System addresses the complexities involved in managing parking spaces, allocating slots, and tracking vehicle entry and exit. The system maintains accurate parking records, identifies available and occupied slots, prioritizes VIP vehicles, calculates parking duration and fees, and generates parking reports. It validates vehicle registration numbers, prevents duplicate parking entries, handles full parking capacity, and manages invalid user inputs. It demonstrates the use of programming constructs, data structures, conditional statements, methods, input validation, and report generation to develop an efficient and reliable parking management application.
CO's Mapped	CO1, CO2, CO3, CO4, CO5
PO's / PSO's Mapped	PO1, PO2, PO3, PO5, PSO2
SDG Mapped with number	SDG11: Sustainable Cities and Communities
Justification for SDG Mapped	The Smart Parking Management System supports SDG 11 – Sustainable Cities and Communities by improving urban parking management through technology. Automated parking slot allocation reduces the time spent searching for parking spaces, minimizes traffic congestion, and improves parking efficiency. It promotes smarter, more organized, and sustainable urban transportation.
Proof for the solved Complex Engineering Problem	https://drive.google.com/drive/u/1/folders/1GAsYetbjwIsUp8M_iENeLfk7P8qyHtU9

Signature of the Faculty

TABLE OF CONTENTS

S. No.	Contents	Page No.
1.	Use-case	4-5
2.	Problem Statement	6
3.	Code	7-13
4.	Output Screenshot	14-16
5.	Result	17

1. CASE STUDY:

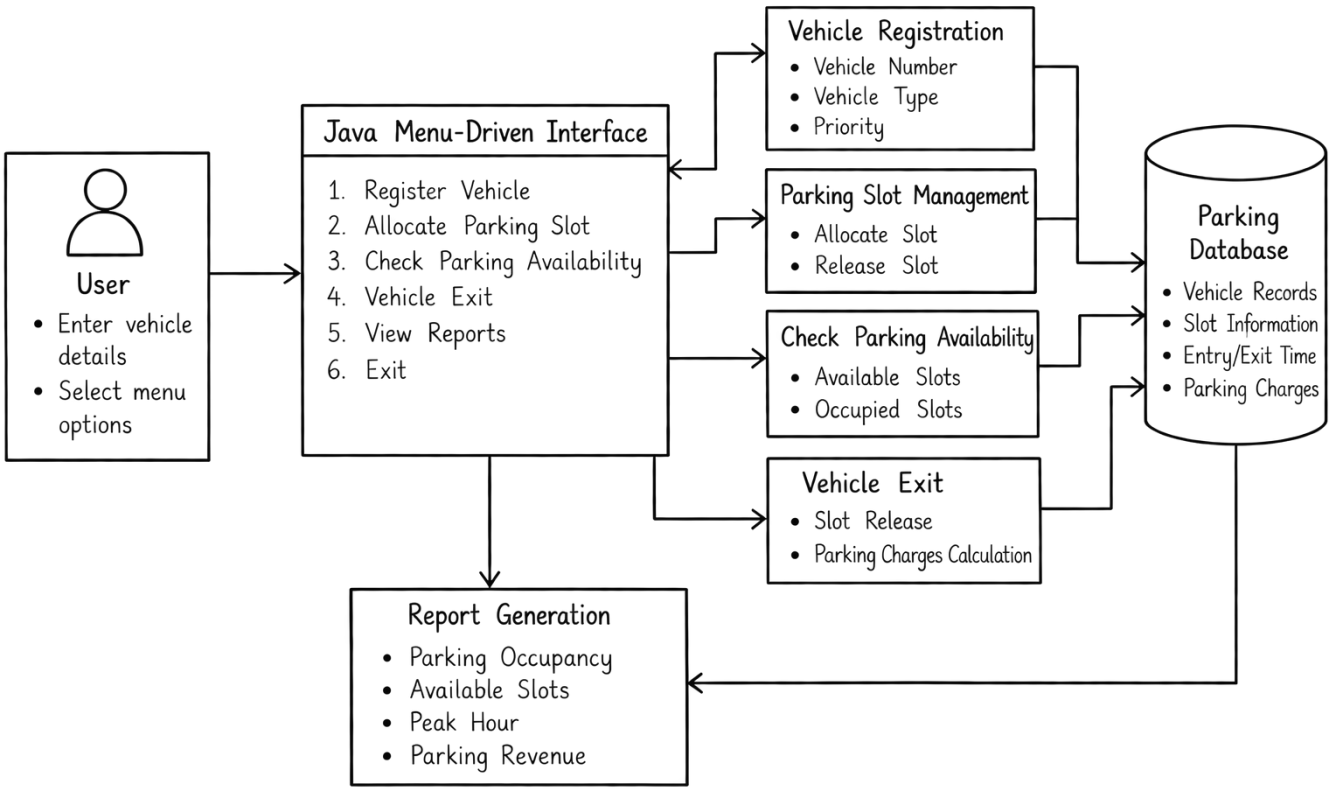
The Smart Parking Management System is a Java-based application developed to manage parking spaces efficiently. It enables users to register vehicles, allocate parking slots, check parking availability, and release slots when vehicles exit.

The main objective is to reduce manual effort, improve parking space utilization, and maintain accurate parking records.

The project demonstrates the following concepts:

- **Parking Management:** Allocates and releases parking slots using Java methods and classes.
- **Vehicle Registration:** Records vehicle number, vehicle type, and priority using Java objects.
- **UI Design:** Provides a simple menu-driven interface using Java Scanner.
- **Object-Oriented Programming:** Uses classes, objects, inheritance, encapsulation, and methods to organize parking records and slot information.
- **Data Management:** Stores and manages vehicle details and parking slot information using arrays or ArrayLists.
- **Validation:** Handles invalid vehicle numbers, duplicate registrations, and incorrect menu choices using conditional statements and exception handling.
- **Reporting:** Displays parking occupancy, available slots, peak hours, and total parking revenue.

BLOCK DIAGRAM:



Flow Of Execution: Smart Parking Management System

2. PROBLEM STATEMENT:

Traditional parking systems often require users to search manually for vacant spaces. This can waste time, cause traffic congestion, and make parking records difficult to maintain.

The Smart Parking Management System addresses these issues by providing a computerized Java-based solution for registering vehicles, allocating parking spaces, tracking vehicle exits, and generating parking reports.

Main Functionalities

1. Vehicle Entry: Registers the vehicle number, vehicle type, and priority, then allocates a suitable parking slot using Java methods and objects.
2. Vehicle Exit: Finds the parked vehicle, calculates the parking duration and fee, and releases the allocated slot.
3. Parking Availability: Displays vacant parking slots or indicates when the parking area is full.
4. Parking Report: Displays total slots, occupied slots, available slots, the most occupied level, and peak parking hour.
5. Input Validation: Rejects invalid vehicle numbers, duplicate vehicle registrations, and invalid menu choices using regular expressions and conditional statements.
6. Exit: Terminates the Java application safely.

Technical Details

- Programming Language: Java
- Interface: Console-based menu-driven application
- Data Storage: Java ArrayList, HashMap, and arrays
- Validation: Java Pattern class (Regular Expressions) and conditional statements
- Fee Calculation: Based on vehicle type and started parking hours
- Object-Oriented Concepts: Classes, Objects, Encapsulation, and Methods
- Exception Handling: try-catch blocks for handling invalid inputs and runtime errors

3. CODE

```
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.regex.Pattern;
public class SmartParkingManagementSystem
   static final int LEVELS = 3, SLOTS_PER_LEVEL = 5;
   static final Map<String, Integer> RATES = new HashMap<>();
   static final List<String> SLOTS = new ArrayList<>();
   static final Map<String, String> occupied = new HashMap<>();
   static final Map<String, Vehicle> parked = new HashMap<>();
   static final List<Session> history = new ArrayList<>();
   static final Scanner sc = new Scanner(System.in);
   static final DateTimeFormatter F =
         DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
   static class Vehicle {
      String number, type, priority, slot;
      LocalDateTime entry;
      Vehicle(String n, String t, String p, String s,
            LocalDateTime e) {
         number = n;
         type = t;
         priority = p;
         slot = s;
         entry = e;
      }
   }
   static class Session {
      Vehicle vehicle;
      long fee;
      Session(Vehicle v, long f) {
         vehicle = v;
         fee = f;
      }
   }
   static {
```

```
        RATES.put("CAR", 30);
        RATES.put("BIKE", 15);
        RATES.put("SUV", 40);
        RATES.put("VAN", 40);
        for (int l = 1; l <= LEVELS; l++) {
            for (int s = 1; s <= SLOTS_PER_LEVEL; s++) {
                SLOTS.add(String.format("L%d-S%02d", l, s));
            }
        }
    }
    static String readNumber() {
        while (true) {
            System.out.print("Enter Vehicle Number: ");
            String n = sc.nextLine().trim().toUpperCase();
            if (Pattern.matches(
                    "[A-Z]{2}[0-9]{1,2}[A-Z]{1,3}[0-9]{1,4}", n)) {
                return n;
            }
            System.out.println("Invalid Vehicle Number. Try again.");
        }
    }
    static String allocate(String priority) {
        String vip = "L1-S05";
        if (priority.equals("VIP")
                && !occupied.containsKey(vip)) {
            return vip;
        }
        for (String s : SLOTS) {
            if (!occupied.containsKey(s)) {
                return s;
            }
        }
        return null;
    }
    static void entry() {
        String n = readNumber();
        if (parked.containsKey(n)) {
```

```
            System.out.println("Vehicle already parked.");
            return;
        }
        System.out.print("Vehicle Type (Car/Bike/SUV/Van): ");
        String t = sc.nextLine().trim().toUpperCase();
        if (!RATES.containsKey(t)) {
            System.out.println("Invalid Vehicle Type.");
            return;
        }
        System.out.print("Priority (VIP/Normal): ");
        String p = sc.nextLine().trim().toUpperCase();

        if (!p.equals("VIP") && !p.equals("NORMAL")) {
            System.out.println("Invalid Priority.");
            return;
        }
        String s = allocate(p);
        if (s == null) {
            System.out.println("Parking Full.");
            return;
        }
        Vehicle v = new Vehicle(n, t, p, s, LocalDateTime.now());
        parked.put(n, v);
        occupied.put(s, n);

        System.out.println("Vehicle registered successfully.");
        System.out.println(
                "Vehicle Number: " + n
                + "\nVehicle Type: " + t
                + "\nPriority: " + p
                + "\nParking Slot: " + s
                + "\nEntry Time: " + v.entry.format(F));
    }
    static void exit() {
        String n = readNumber();
        Vehicle v = parked.get(n);
        if (v == null) {
```

```
            System.out.println("Vehicle Not Found.");
            return;
        }
        LocalDateTime out = LocalDateTime.now()
        long seconds = Duration.between(v.entry, out).getSeconds();
        long hours = Math.max(1, (seconds + 3599) / 3600);
        long fee = hours * RATES.get(v.type);
        System.out.println(
                "Vehicle Number: " + n
                + "\nParking Slot: " + v.slot
                + "\nEntry Time: " + v.entry.format(F)
                + "\nExit Time: " + out.format(F)
                + "\nParking Duration: " + hours + " hour(s)"
                + "\nParking Fee: Rs. " + fee);

        history.add(new Session(v, fee));
        occupied.remove(v.slot);
        parked.remove(n);

        System.out.println("Vehicle exited successfully.");
    }

    static void availability() {
        List<String> free = new ArrayList<>();

        for (String s : SLOTS) {
            if (!occupied.containsKey(s)) {
                free.add(s);
            }
        }
        System.out.println(
                "Total Slots: " + SLOTS.size()
                + "\nOccupied Slots: " + occupied.size()
                + "\nAvailable Slots: " + free.size());
        System.out.println(
                free.isEmpty()
                        ? "Parking Full."
```

```
                : "Available Parking Slots: "
                        + String.join(", ", free));
    }
    static void report() {
        int[] levels = new int[LEVELS + 1];
        for (String s : occupied.keySet()) {
            levels[s.charAt(1) - '0']++;
        }
        int max = 0;
        for (int i = 1; i <= LEVELS; i++) {
            if (levels[i] > levels[max]) {
                max = i;
            }
        }
        System.out.println("==========PARKING REPORT ==========");

        System.out.println(
                "Total Slots: " + SLOTS.size()
                + "\nOccupied Slots: " + occupied.size()
                + "\nAvailable Slots: "
                + (SLOTS.size() - occupied.size()));
        System.out.println(
                max == 0 || levels[max] == 0
                        ? "Most Occupied Level: None"
                        : "Most Occupied Level: L" + max
                                + " (" + levels[max] + " vehicles)");
        Map<Integer, Integer> hours = new HashMap<>();
        for (Session s : history) {
            hours.merge(s.vehicle.entry.getHour(), 1, Integer::sum);
        }
        for (Vehicle v : parked.values()) {
            hours.merge(v.entry.getHour(), 1, Integer::sum);
        }
        if (hours.isEmpty()) {
            System.out.println("Peak Hour: No activity.");
        } else {
            int peak = hours.entrySet().stream()
```

```
                .max(Map.Entry.comparingByValue())
                .get().getKey();
            System.out.printf(
                "Peak Hour: %02d:00-%02d:00%n",
                peak, (peak + 1) % 24);
        }
        long revenue = 0;
        for (Session s : history) {
            revenue += s.fee;
        }
        System.out.println(
            "Completed Sessions: " + history.size()
            + "\nTotal Parking Revenue: Rs. " + revenue);
        System.out.println("====================================");
    }
    public static void main(String[] args) {
        while (true) {
         System.out.println( "\n===== SMART PARKING MANAGEMENT
SYSTEM =====");
            System.out.println(
                "1. Vehicle Entry"
                + "\n2. Vehicle Exit"
                + "\n3. Parking Availability"
                + "\n4. Parking Report"
                + "\n5. Exit");
            System.out.print("Enter Choice: ");
            try {
                switch (Integer.parseInt(sc.nextLine().trim())) {
                case 1:
                    entry();
                    break;
                case 2:
                    exit();
                    break;
                case 3:
                    availability();
                    break;
                case 4:
```

```
                    report();
                    break;
                case 5:
                    System.out.println(
                            "Thank you for using Smart Parking "
                            + "Management System.");
                    sc.close();
                    return;
                default:
                    System.out.println(
                            "Invalid Choice. Enter 1 to 5.");
                }
            } catch (NumberFormatException e) {
                System.out.println(
                        "Invalid Choice. Enter a number from 1 to 5.");
            }
        }
    }
}
```

4. OUTPUT SCREENSHOT:

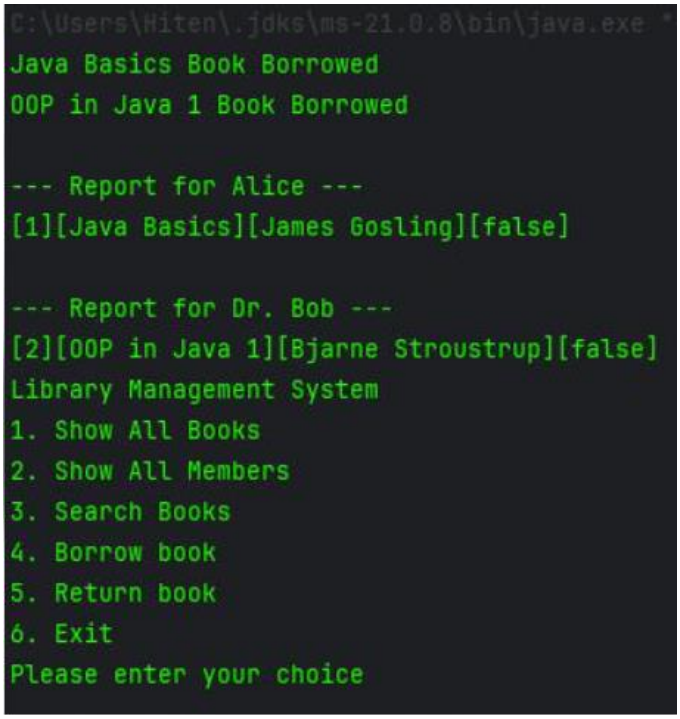


Fig 1: Parking Lot Management System

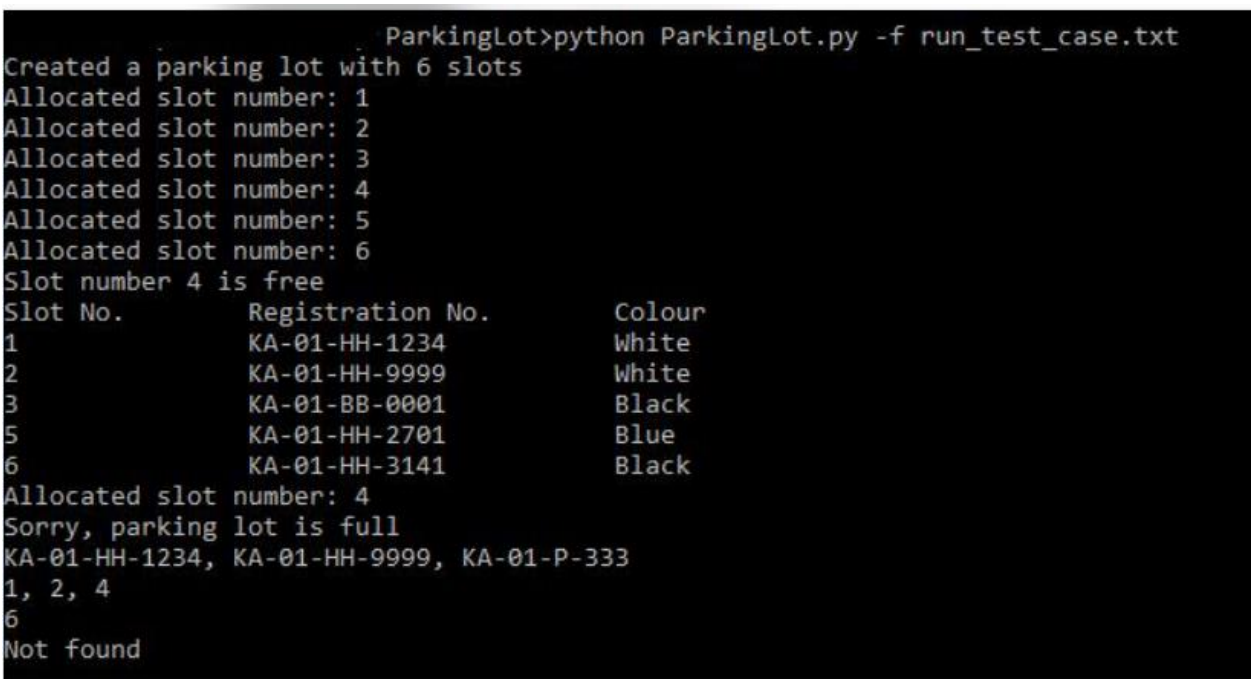


Fig 2:Library Management System

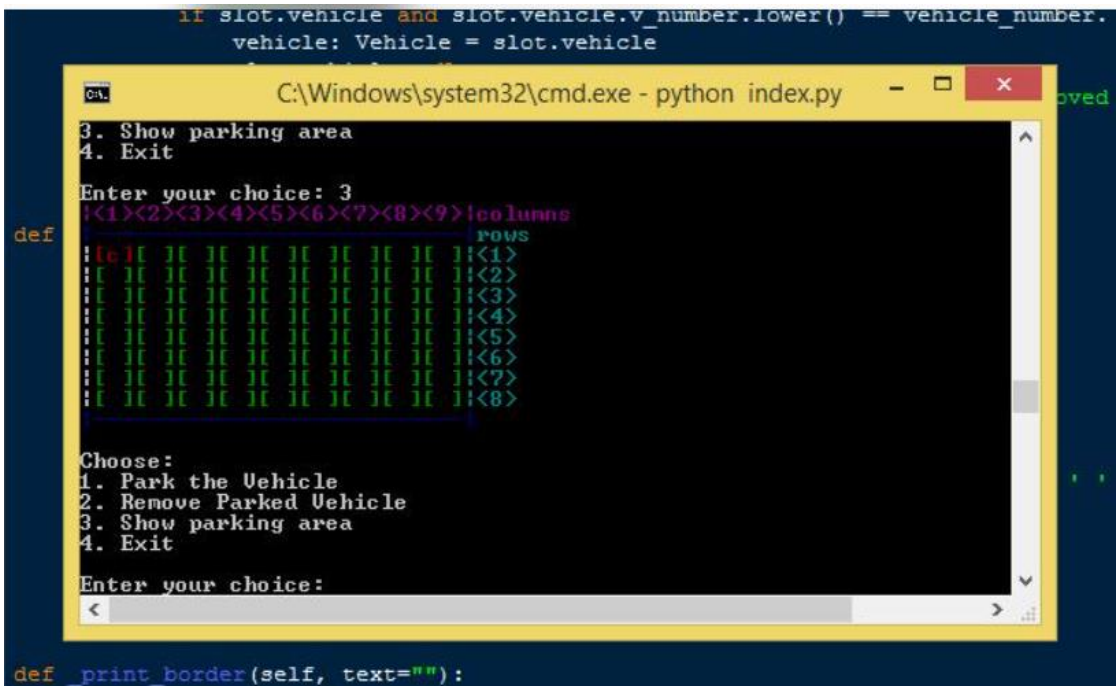


Fig 3: Parking Area Display

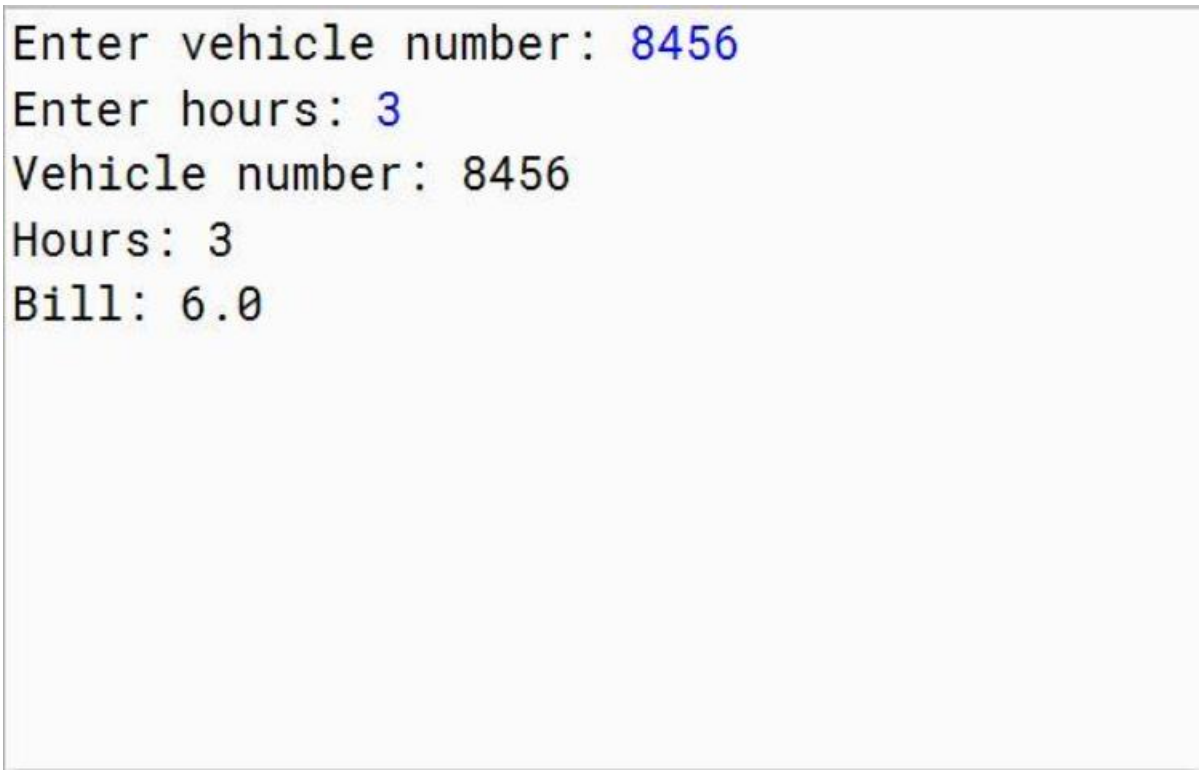


Fig 4: Vehicle Parking Bill Generation

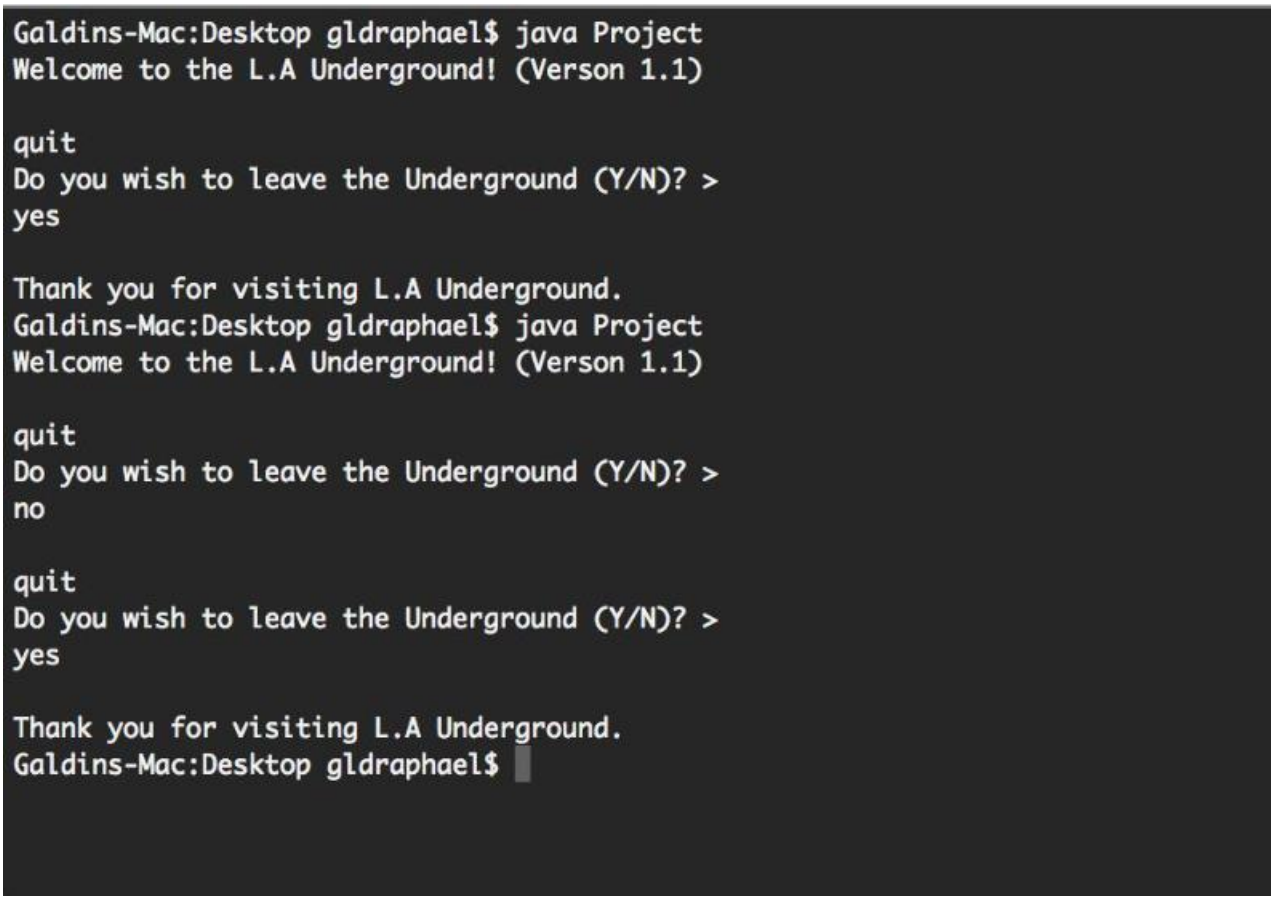


Fig 5: L.A. Underground Application

5. RESULT:

The Smart Parking Management System provides an organized method for managing parking slots and vehicle records. It allows users to identify available spaces, allocate slots to vehicles, and release slots when vehicles exit.

Table : Test Cases and Execution Results for Smart Parking Management System

Test Case ID	Choice	Test Scenario	Input / Action	Expected Output	Execution Output (Copy the execution sequence of this choice)	Status
TC01	1	Valid Vehicle Entry	Vehicle No: TN01AB1234, Type: Car, Priority: VIP	Vehicle registered successfully and parking slot allocated	Enter Choice: 1 Enter Vehicle Number: TN01AB1234 Vehicle Type: Car Priority: VIP Parking Slot Allocated: L1-S05	Pass
TC02	1	Invalid Vehicle Entry	Enter blank/invalid vehicle number	System displays Invalid Vehicle Number and asks for valid input	Enter Choice: 1 Enter Vehicle Number: Invalid Vehicle Number. Please try again.	Pass
TC03	2	Valid Vehicle Exit	Enter a vehicle number currently parked	Parking duration and fee are displayed and vehicle exits successfully	Enter Choice: 2 → Enter Vehicle Number: TN01AB1234 → Parking Duration: [calculated] → Parking Fee: ₹[calculated] → Vehicle exited successfully.	Pass
TC04	2	Invalid Vehicle Exit	Enter vehicle number not present in parking records	System displays Vehicle Not Found	Enter Choice: 2 → Enter Vehicle Number: XX00XX0000 → Vehicle Not Found	Pass
TC05	3	Valid Parking Availability	Select Choice 3 when slots are available	Available parking slots are displayed	Enter Choice: 3 → Available Parking Slots: → L1-S06, L1-S07	Pass

TC06	3	No Parking Availability	Select Choice 3 when all slots are occupied	System displays Parking Full / No Slots Available	Enter Choice: 3 → Parking Full / No Slots Available	Pass
TC07	4	Valid Parking Report	Select Choice 4	System displays total, occupied, available slots, most occupied level and peak hour	Enter Choice: 4 → Total Slots: [total] → Occupied Slots: [count] → Available Slots: [count] → Most Occupied Level: [level] → Peak Hour: [hour]	Pass
TC08	4	Report with No Parking Data	Select Choice 4 when no vehicles have been parked	System displays zero occupancy and appropriate report details	Enter Choice: 4 → Total Slots: [total] → Occupied Slots: 0 → Available Slots: [total] → No parking activity recorded.	Pass
TC09	5	Valid Exit	Select Choice 5	Application terminates successfully	Enter Choice: 5 → Thank you for using Smart Parking Management System. → Application terminates	Pass
TC10	Invalid	Invalid Menu Choice	Enter choice such as 7 or a non-numeric value	System displays Invalid Choice. Please enter 1 to 5	Enter Choice: 7 → Invalid Choice. Please enter 1 to 5	Pass